TÀI LIỆU TÓM TẮT C++
DÀNH CHO HỌC SINH ĐỘI TUYỂN HSG LỚP 9

1. Khung chương trình C++ và vào/ra dữ liệu

Một chương trình C++ thi đấu thường gồm khai báo thư viện, khai báo biến/hàm, hàm main(), đọc dữ liệu, xử lý và in kết quả. Với hệ thống chấm bài, chương trình cần đọc đúng input và in đúng output, không in thừa lời nhắc.

Tóm tắt cú pháp và công dụng

Thành phần	Cú pháp thường dùng	Công dụng
Thư viện	#include <bits/stdc++.h>	Gọi hầu hết thư viện hay dùng trong lập trình thi đấu.
Không gian tên	using namespace std;	Giúp dùng cin, cout, string, sort, stack... không cần tiền tố std::.
Hàm main	int main(){ ... return 0; }	Điểm bắt đầu thực thi chương trình.
Nhập dữ liệu	cin >> a >> b;	Đọc dữ liệu từ bàn phím hoặc file input.
Xuất dữ liệu	cout << kq << "\n";	In kết quả ra màn hình hoặc file output.
Đọc/ghi file	freopen("BAI.INP","r",stdin);	Chuyển cin/cout sang file khi đề yêu cầu.

Ví dụ minh họa

```
#include <bits/stdc++.h>
using namespace std;
long long a, b;
int main() {
    // freopen("BAITAP.INP", "r", stdin);
    // freopen("BAITAP.OUT", "w", stdout);
    cin >> a >> b;
    cout << a + b << "\n";
    return 0;
}
```

Ghi nhớ: Khi nộp bài online, thường không dùng freopen nếu hệ thống yêu cầu đọc từ stdin và ghi ra stdout. Khi luyện Themis theo tên file, có thể bật freopen theo đúng tên file đề bài.

2. Kiểu dữ liệu, biến, hằng và phép toán

Kiểu dữ liệu quyết định miền giá trị và cách máy tính lưu trữ dữ liệu. Trong thi đấu, chọn sai kiểu có thể gây tràn số. Với số lớn tới 10^18 nên dùng long long.

Tóm tắt cú pháp và công dụng

Nội dung	Cú pháp/Ví dụ	Công dụng
Số nguyên	int n; long long s;	Lưu số nguyên. int khoảng 2*10^9, long long khoảng 9*10^18.
Số thực	double x;	Lưu số thập phân; dùng khi cần kết quả

		thực.
Ký tự	char c;	Lưu một ký tự như A, b, 7.
Logic	bool ok;	Lưu true/false.
Hằng	#define N 100000 hoặc const int N=100000;	Lưu giá trị cố định.
Phép toán số học	+, -, *, /, %	Tính toán; / với số nguyên là chia lấy nguyên, % là chia lấy dư.
Hàm toán học	sqrt(x), abs(x), min(a,b), max(a,b)	Tính căn, trị tuyệt đối, nhỏ nhất, lớn nhất.

Ví dụ minh họa

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    long long a, b;
    cin >> a >> b;
    cout << "Tong = " << a + b << "\n";
    cout << "Thuong nguyen = " << a / b << "\n";
    cout << "Du = " << a % b << "\n";
    cout << "Max = " << max(a, b) << "\n";
    return 0;
}
```

*Ghi nhớ: Với biểu thức nhân hai số có thể lớn, nên ép kiểu hoặc khai báo long long ngay từ đầu: long long s = 1LL*a*b;*

Bài tập ứng dụng

1. **Chia kẹo:** Có n cái kẹo chia đều cho m bạn. In mỗi bạn nhận bao nhiêu cái và còn dư bao nhiêu cái.
2. **Chữ số cuối:** Nhập n. In chữ số cuối của n.
3. **Tổng hai chữ số cuối:** Nhập n. In tổng của hai chữ số cuối cùng của n.
4. **Làm tròn điểm:** Nhập điểm thực x. In x với 2 chữ số sau dấu phẩy.
5. **Khoảng cách trên trục số:** Nhập a, b. In |a-b|.

3. Cấu trúc rẽ nhánh if, if-else, switch

Rẽ nhánh dùng để xử lý bài toán có nhiều trường hợp. Điều kiện thường được tạo bởi phép so sánh và phép logic.

Tóm tắt cú pháp và công dụng

Nội dung	Cú pháp/Ví dụ	Công dụng
So sánh	>, <, >=, <=, ==, !=	Tạo điều kiện đúng/sai.
Logic	&&, ||, !	Kết hợp nhiều điều kiện.

if	if (dk) lenh;	Thực hiện lệnh khi điều kiện đúng.
if-else	if (dk) lenh1; else lenh2;	Chọn một trong hai nhánh.
if lồng nhau	if (...) ... else if (...) ... else ...	Phân loại nhiều trường hợp.
switch	switch(x){case 1: ...; break; default: ...;}	Rẽ nhánh theo giá trị rời rạc.

Ví dụ minh họa

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    long long a, b, c;
    cin >> a >> b >> c;
    long long mx = a;
    if (b > mx) mx = b;
    if (c > mx) mx = c;
    cout << mx << "\n";
    if (a + b > c && a + c > b && b + c > a)
        cout << "YES\n";
    else
        cout << "NO\n";
    return 0;
}
```

Ghi nhớ: Lỗi thường gặp: dùng = thay vì == trong điều kiện; đặt dấu ; ngay sau if; thiếu ngoặc {} khi có nhiều lệnh trong một nhánh.

Bài tập ứng dụng

6. **Chẵn lẻ:** Nhập n. In EVEN nếu n chẵn, ODD nếu n lẻ.
7. **Số lớn nhất trong ba số:** Nhập a, b, c. In số lớn nhất.
8. **Kiểm tra tam giác:** Nhập a, b, c. Kiểm tra có tạo thành tam giác không.
9. **Xếp loại điểm:** Nhập điểm x. In Giỏi/Khá/Trung bình/Yếu theo thang điểm cho trước.
10. **Ngày trong tuần:** Nhập số k từ 1 đến 7. Dùng switch in tên thứ tương ứng.

4. Vòng lặp for và tư duy lặp

Vòng lặp for phù hợp khi biết trước số lần lặp, ví dụ duyệt i từ 1 đến n, in bảng cửu chương, tính tổng dãy số.

Tóm tắt cú pháp và công dụng

Nội dung	Cú pháp/Ví dụ	Công dụng

for tăng	for (int i=1;i<=n;i++)	Duyệt i tăng dần.
for giảm	for (int i=n;i>=1;i--)	Duyệt i giảm dần.
for lồng nhau	for (...) for (...)	Duyệt cặp chỉ số, bảng, ma trận.
break	break;	Thoát khỏi vòng lặp gần nhất.
continue	continue;	Bỏ qua phần còn lại của lần lặp hiện tại.

Ví dụ minh họa

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    int n;
    cin >> n;
    long long s = 0;
    for (int i = 1; i <= n; i++) s += i;
    cout << s << "\n";
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= i; j++) cout << "*";
        cout << "\n";
    }
    return 0;
}
```

Ghi nhớ: Độ phức tạp: một vòng for từ 1 đến n thường là O(n); hai vòng for lồng nhau thường là O(n^2). Với n lớn, cần tránh O(n^2) nếu không cần thiết.

Bài tập ứng dụng

11. **Tổng 1 đến n:** Nhập n. Tính 1+2+...+n bằng vòng lặp.
12. **Tổng số chẵn:** Nhập n. Tính tổng các số chẵn từ 1 đến n.
13. **Đếm ước:** Nhập n. Đếm số ước dương của n.
14. **Bảng cửu chương:** Nhập k. In bảng nhân k từ 1 đến 10.
15. **Tam giác sao:** Nhập n. In tam giác sao gồm n dòng.

5. Vòng lặp while, do-while và xử lý chữ số

while phù hợp khi chưa biết trước số lần lặp, ví dụ tách chữ số, tìm UCLN, lặp đến khi điều kiện sai. do-while luôn thực hiện thân vòng lặp ít nhất một lần.

Tóm tắt cú pháp và công dụng

Nội dung	Cú pháp/Ví dụ	Công dụng

while	while (dk) { ... }	Lặp khi điều kiện còn đúng.
do-while	do { ... } while (dk);	Làm trước, kiểm tra sau.
Tách chữ số	d = n % 10; n /= 10;	Lấy và bỏ chữ số cuối.
UCLN Euclid	while (b!=0){r=a%b; a=b; b=r;}	Tính ước chung lớn nhất nhanh.
Tránh lặp vô hạn	cần lệnh làm thay đổi điều kiện	Bảo đảm vòng lặp dừng.

Ví dụ minh họa

```
#include <bits/stdc++.h>
using namespace std;
long long gcdll(long long a, long long b) {
    while (b != 0) {
        long long r = a % b;
        a = b;
        b = r;
    }
    return a;
}
int main() {
    long long n;
    cin >> n;
    long long s = 0, x = n;
    while (x > 0) {
        s += x % 10;
        x /= 10;
    }
    cout << s << "\n";
    return 0;
}
```

Ghi nhớ: Với n=0, vòng while(n>0) không chạy. Nếu bài yêu cầu xử lý số 0, cần xét riêng hoặc dùng do-while.

Bài tập ứng dụng

16. **Tổng chữ số:** Nhập n. Tính tổng các chữ số của n.
17. **Đảo số:** Nhập n. In số tạo bởi các chữ số của n theo thứ tự ngược lại.
18. **Đếm chữ số:** Nhập n. Đếm số chữ số của n.

19. **UCLN và BCNN:** Nhập a, b. In UCLN và BCNN của hai số.
20. **Số nguyên tố gần nhất:** Nhập n. Tìm số nguyên tố nhỏ nhất không nhỏ hơn n.

6. Mảng một chiều

Mảng một chiều dùng để lưu một dãy nhiều phần tử cùng kiểu. Trong C++, mảng thường bắt đầu từ chỉ số 0, nhưng khi dạy học sinh có thể dùng từ 1 đến n để gần với ký hiệu toán học.

Tóm tắt cú pháp và công dụng

Nội dung	Cú pháp/Ví dụ	Công dụng
Khai báo	int a[100005];	Tạo mảng số nguyên.
Truy xuất	a[i]	Lấy hoặc gán phần tử thứ i.
Nhập mảng	for (int i=1;i<=n;i++) cin>>a[i];	Đọc n phần tử.
Duyệt mảng	for (int i=1;i<=n;i++)	Xử lý từng phần tử.
Sắp xếp	sort(a+1,a+n+1);	Sắp tăng dần đoạn a[1..n].
Khởi tạo 0	memset(a,0,sizeof(a));	Đặt các phần tử về 0.

Ví dụ minh họa

```
#include <bits/stdc++.h>
using namespace std;
const int N = 100005; long long a[N];
int main() {
    int n;
    cin >> n;
    for (int i = 1; i <= n; i++) cin >> a[i];
    long long s = 0;
    int dem = 0;
    for (int i = 1; i <= n; i++) {
        if (a[i] > 0) s += a[i];
        if (a[i] % 2 != 0) dem++;
    }
    cout << s << " " << dem << "\n";
    return 0;
}
```

Ghi nhớ: Không truy xuất ngoài biên như a[-1], a[n+1]. Với mảng lớn nên khai báo toàn cục để tránh tràn bộ nhớ stack.

Bài tập ứng dụng

21. **Tổng dãy:** Nhập n và dãy a. Tính tổng tất cả phần tử.

22. **Tổng dương:** Nhập n và dãy a. Tính tổng các phần tử dương.
23. **Đếm số lẻ:** Nhập n và dãy a. Đếm số phần tử lẻ.
24. **Tìm min/max:** Nhập n và dãy a. In giá trị nhỏ nhất và lớn nhất.
25. **Sắp xếp tăng:** Nhập n và dãy a. In dãy sau khi sắp xếp tăng dần.

7. Một số kỹ thuật cơ bản trên mảng một chiều

Sau khi biết nhập và duyệt mảng, học sinh cần luyện các kỹ thuật thường gặp: đếm theo điều kiện, tìm vị trí, tính tổng, sắp xếp, đếm cặp, tìm khoảng cách nhỏ nhất.

Tóm tắt cú pháp và công dụng

Kỹ thuật	Ý tưởng	Độ phức tạp
Duyệt kiểm tra	Duyệt từng a[i], nếu thỏa thì xử lý.	O(n)
Tìm vị trí đầu tiên	Gặp phần tử thỏa thì lưu vị trí và có thể break.	O(n)
Sắp xếp rồi xử lý	Sắp xếp để các phần tử gần nhau đứng cạnh nhau.	O(n log n)
Đếm cặp trâu	Duyệt mọi i<j.	O(n^2)
Tìm min |a[i]-a[j]|	Sắp xếp, chỉ cần xét hai phần tử kề nhau.	O(n log n)

Ví dụ minh họa

```
#include <bits/stdc++.h>
using namespace std;
const int N = 100005; long long a[N];
int main() {
    int n;
    cin >> n;
    for (int i = 1; i <= n; i++) cin >> a[i];
    sort(a + 1, a + n + 1);
    long long ans = llabs(a[2] - a[1]);
    for (int i = 2; i < n; i++) {
        ans = min(ans, llabs(a[i + 1] - a[i]));
    }
    cout << ans << "\n";
    return 0;
}
```

Ghi nhớ: Bài đếm cặp đảo nghịch i<j, a[i]>a[j] làm trâu O(n^2) phù hợp n nhỏ. Với n lớn cần học thêm merge sort/Fenwick, nhưng giai đoạn đầu nên cho học sinh hiểu bản chất trước.

Bài tập ứng dụng

26. **Vị trí số lớn nhất:** Nhập dãy a. In vị trí đầu tiên của phần tử lớn nhất.
27. **Tổng trị tuyệt đối:** Nhập dãy a. Tính tổng |a[i]|.
28. **Đếm cặp nghịch thế:** Đếm số cặp i<j sao cho a[i]>a[j] với n nhỏ.
29. **Hai số gần nhau nhất:** Tìm giá trị nhỏ nhất của |a[i]-a[j]| với i khác j.
30. **Loại trùng sau sắp xếp:** In các giá trị khác nhau của dãy theo thứ tự tăng dần.

8. Mảng hai chiều và ma trận

Mảng hai chiều biểu diễn bảng số/ma trận gồm hàng và cột. Cần hai chỉ số i, j và thường dùng hai vòng for lồng nhau để nhập, duyệt, tính toán.

Tóm tắt cú pháp và công dụng

Nội dung	Cú pháp/Ví dụ	Công dụng
Khai báo	int a[105][105];	Tạo ma trận.
Truy xuất	a[i][j]	Phần tử hàng i, cột j.
Nhập ma trận	for i, for j, cin>>a[i][j];	Đọc dữ liệu bảng.
Đường chéo chính	a[i][i]	Áp dụng với ma trận vuông.
Đường chéo phụ	a[i][n+1-i]	Áp dụng với ma trận vuông.
Tổng hàng/cột	cố định i hoặc j rồi cộng	Xử lý thống kê trên ma trận.

Ví dụ minh họa

```
#include <bits/stdc++.h>
using namespace std;
const int N = 105; int a[N][N];
int main() {
    int m, n;
    cin >> m >> n;
    long long s = 0;
    for (int i = 1; i <= m; i++)
        for (int j = 1; j <= n; j++) {
            cin >> a[i][j];
            s += a[i][j];
        }
    cout << s << "\n";
    return 0;
}
```

*Ghi nhớ: Với ma trận m*n, duyệt toàn bộ phần tử có độ phức tạp O(m*n). Cần chú ý thứ tự chỉ số hàng trước, cột sau.*

Bài tập ứng dụng

31. **Tổng ma trận:** Nhập m, n và ma trận. In tổng các phần tử.
32. **Tổng từng hàng:** In tổng của từng hàng trong ma trận.
33. **Hàng có tổng lớn nhất:** Tìm chỉ số hàng có tổng lớn nhất.
34. **Tổng đường chéo chính:** Nhập ma trận vuông n*n. Tính tổng đường chéo chính.
35. **Đếm số chẵn trong ma trận:** Đếm các phần tử chẵn của ma trận.

9. Hàm trong C++

Hàm giúp chia bài toán lớn thành các phần nhỏ, dễ đọc và dễ kiểm tra. Trong lập trình thi đấu, hàm thường dùng để kiểm tra nguyên tố, tính UCLN, tính lũy thừa, xử lý một công việc lặp lại nhiều lần.

Tóm tắt cú pháp và công dụng

Nội dung	Cú pháp/Ví dụ	Công dụng
Khai báo hàm	kiểu_trả_về tên_hàm(tham_số)	Định nghĩa chức năng riêng.
return	return giá_trị;	Trả kết quả về nơi gọi hàm.
Hàm bool	bool nt(long long n)	Trả đúng/sai, phù hợp kiểm tra.
Tham trị	void f(int x)	Hàm nhận bản sao, thay đổi x không ảnh hưởng bên ngoài.
Tham biến	void swap2(int &a,int &b)	Thay đổi trực tiếp biến truyền vào.
Top-down	main gọi các hàm nhỏ	Tổ chức chương trình rõ ràng.

Ví dụ minh họa

```
#include <bits/stdc++.h>
using namespace std;
bool nt(long long n) {
    if (n < 2) return false;
    for (long long i = 2; i * i <= n; i++)
        if (n % i == 0) return false;
    return true;
}
long long ucln(long long a, long long b) {
    while (b != 0) {
        long long r = a % b;
        a = b;
        b = r;
    }
```

```
    return a;
}
int main() {
    long long n;
    cin >> n;
    if (nt(n)) cout << "YES\n";
    else cout << "NO\n";
    return 0;
}
```

*Ghi nhớ: Trong hàm kiểm tra nguyên tố, nên dùng điều kiện i*i<=n để tránh gọi sqrt nhiều lần; nếu n rất lớn, cần lưu ý i*i có thể tràn, có thể dùng i<=n/i.*

Bài tập ứng dụng

36. **Hàm kiểm tra nguyên tố:** Viết hàm nt(n), nhập n, in YES nếu n nguyên tố.
37. **Đếm nguyên tố trong đoạn:** Viết hàm nt(n), nhập a, b, đếm số nguyên tố trong [a,b].
38. **Hàm UCLN:** Viết hàm ucln(a,b), nhập nhiều cặp số và in UCLN.
39. **Hàm tổng chữ số:** Viết hàm tongcs(n), trả về tổng chữ số của n.
40. **Hàm đổi chỗ:** Viết hàm swap2(int &a,int &b), kiểm tra sau lời gọi hai biến đã đổi giá trị.

10. Kiểu dữ liệu string

string dùng để xử lý xâu ký tự. Xâu giống mảng ký tự, chỉ số bắt đầu từ 0. Đây là phần rất quan trọng trong các bài toán xử lý văn bản, mật mã, đếm ký tự, đối xứng.

Tóm tắt cú pháp và công dụng

Nội dung	Cú pháp/Ví dụ	Công dụng
Khai báo	string s, t;	Tạo biến xâu.
Nhập không dấu cách	cin >> s;	Đọc một từ.
Nhập có dấu cách	getline(cin, s);	Đọc cả dòng.
Độ dài	s.size() hoặc s.length()	Số ký tự của xâu.
Truy cập	s[i]	Ký tự thứ i, từ 0 đến size-1.
Ghép xâu	s + t	Nối hai xâu.
Xâu con	s.substr(pos, len)	Lấy len ký tự từ vị trí pos.
Tìm kiếm	s.find(t)	Tìm vị trí xuất hiện của t trong s.
Xóa/chèn	erase, insert	Chỉnh sửa nội dung xâu.

Ví dụ minh họa

```
#include <bits/stdc++.h>
using namespace std;
```

```
int main() {
    string s;
    cin >> s;
    int dem = 0;
    for (int i = 0; i < (int)s.size(); i++) {
        if (s[i] >= '0' && s[i] <= '9') dem++;
    }
    cout << dem << "\n";

    string t = s;
    reverse(t.begin(), t.end());
    if (s == t) cout << "PALINDROME\n";
    else cout << "NO\n";
    return 0;
}
```

Ghi nhớ: Sau khi dùng cin >> n, nếu cần getline ngay sau đó, cần gọi cin.ignore(); để bỏ ký tự xuống dòng còn lại trong bộ đệm.

Bài tập ứng dụng

41. **Đếm chữ số trong xâu:** Nhập xâu s. Đếm số ký tự là chữ số.
42. **Đếm chữ cái hoa:** Nhập xâu s. Đếm số ký tự từ A đến Z.
43. **Xâu đối xứng:** Kiểm tra xâu s có phải palindrome không.
44. **Đảo xâu:** Nhập xâu s. In xâu đảo ngược.
45. **Tìm xâu con:** Nhập s và t. Kiểm tra t có xuất hiện trong s không.

11. Cấu trúc dữ liệu stack

Stack là ngăn xếp hoạt động theo nguyên tắc vào sau ra trước LIFO. Phần tử thêm sau cùng sẽ được lấy ra trước. Stack thường dùng trong kiểm tra ngoặc đúng, đảo dữ liệu, mô phỏng thao tác, xóa ký tự theo quy tắc.

Tóm tắt cú pháp và công dụng

Nội dung	Cú pháp/Ví dụ	Công dụng
Khai báo	stack<int> st; stack<char> s;	Tạo stack lưu số hoặc ký tự.
empty	st.empty()	Kiểm tra stack rỗng.
size	st.size()	Số phần tử hiện có.
push	st.push(x);	Đưa x lên đỉnh stack.
top	st.top();	Lấy giá trị phần tử ở đỉnh.
pop	st.pop();	Xóa phần tử ở đỉnh; không trả về giá trị.

Ví dụ minh họa

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    string s;
    cin >> s;
    stack<char> st;
    bool ok = true;
    for (int i = 0; i < (int)s.size(); i++) {
        if (s[i] == '(') st.push(s[i]);
        else if (s[i] == ')') {
            if (st.empty()) ok = false;
            else st.pop();
        }
    }
    if (!st.empty()) ok = false;
    cout << (ok ? "YES" : "NO") << "\n";
    return 0;
}
```

Ghi nhớ: Không được gọi top() hoặc pop() khi stack đang rỗng. Luôn kiểm tra empty() trước.

Bài tập ứng dụng

46. **Đảo dãy bằng stack:** Nhập n số, dùng stack để in lại dãy theo thứ tự ngược.
47. **Ngoặc tròn đúng:** Nhập xâu chỉ gồm (và). Kiểm tra dãy ngoặc đúng.
48. **Ngoặc nhiều loại:** Kiểm tra xâu gồm (), [], {} có hợp lệ không.
49. **Xóa ký tự KDEL:** Nhập xâu số và k. Xóa k chữ số để tạo số nhỏ nhất hoặc lớn nhất theo yêu cầu.
50. **Mô phỏng lệnh push/pop:** Nhập q truy vấn PUSH x, POP, TOP. Thực hiện bằng stack và in kết quả khi cần.